0) Last minute Memory Grid

oprule Topic	Key formula / exam line
Clos	$r = N/n$; strict nonblocking: $m \geq 2n-1$; rearrangeable: $m \geq n$
Fat-tree	Core: $(k/2)^2$; total hosts: $k^3/4$; total switches: $5k^2/4$
RAID reliability	RAID0: $P_{loss} \approx np$; RAID1: $P_{loss} \approx (n/2)p^2$; RAID5: $\binom{n}{2}p^2$; RAID6: $\binom{n}{3}p^3$
RAID capacity	RAID0: nS; RAID1: $(n/2)S$; RAID5: $(n-1)S$; RAID6: $(n-2)S$
XOR recovery	$P = D_1 \oplus \cdots \oplus D_{n-1}$, and $D_i = P \oplus$(all other surviving D blocks)
EDF/SEDF	$U = \sum_i s_i/p_i$; schedulable if $U \leq 1$; hyperperiod $H = \text{lcm}(p_1, \ldots, p_n)$
Credit scheduler	$share_i = w_i / \sum_j w_j$; $C_i = C \cdot w_i / \sum_j w_j$
AIMD	$STT = \frac{W}{2}RTT$; $p = \frac{8}{3W^2}$; $T \approx 1.22 \cdot \frac{MTU}{RTT\sqrt{p}}$
Diffie-Hellman	Shared secret: $s = g^{ab} \bmod p$
RSA signature	Sign: $S = M^d \bmod n$; verify: $M' = S^e \bmod n$
CAP	Under partition, choose CP (correctness) vs AP (availability) based on business priority
PageRank	$r_i = \sum_{j \to i} r_j/d_j$; damped: $\mathbf{r} = dM\mathbf{r} + (1-d)\frac{1}{n}\mathbf{1}$
Consistent hashing	Clockwise successor rule; load-skew risk: $P = n/2^{n-1}$; rebalancing is localized

1) Virtualization

Aspect	Full Virtualization	Para-virtualization	Hardware-assisted Virtualization
Core mechanism	Trap-and-emulate / binary translation of sensitive instructions	Guest OS is modified to use hypercalls to the hypervisor	CPU virtualization extensions (Intel VT-x / AMD-V) trap sensitive instructions in hardware
Guest OS modification	Not required	Required	Not required
OS compatibility	High (works with unmodified guest OSes)	Lower (only modified/ported guest OSes)	High (unmodified guest OSes supported)
Typical performance	Lowest of the three in legacy software-only implementations	Better than full virtualization due to reduced translation overhead	Best practical performance in modern cloud deployments
Main advantages	Broad compatibility; strong isolation model; easy to run legacy images	Lower virtualization overhead; faster I/O with PV drivers; simpler emulation path	Near-native speed; strong compatibility; hardware-enforced isolation (for example EPT/SLAT, IOMMU)
Main disadvantages	Binary translation and emulation overhead; higher hypervisor complexity	Guest kernel changes and maintenance burden; weaker portability across hypervisors	Requires VT-x/AMD-V capable hardware; VM-exit overhead can still hurt some trap-heavy workloads
Typical use case	Compatibility-focused environments and legacy workload support	Controlled environments where guest OS can be customized	Mainstream data-center/cloud virtualization

Quick memory line: compatibility ranking is Hardware-assisted approximately Full greater than Para; performance ranking is Hardware-assisted greater than Para greater than Full.

2) Clos and Fat-Tree

Setup / Formula

Aspect	3-stage Clos	Fat-tree (k-port)
Key parameters	N = total input ports; n = ports per ingress switch; k = number of middle switches; m = middle switch port count	k = port count of every switch (same k used everywhere)
Ingress / egress switches	$r = N/n$ switches per stage	$k/2$ edge switches per pod; $k/2$ aggregation switches per pod
Middle switches	k switches, size $m \times m$	$(k/2)^2$ core switches
Switch sizes	Stage 1: $n \times k$; Stage 2: $m \times m$; Stage 3: $k \times n$	Edge: k-port; Aggregation: k-port; Core: k-port
Pods	—	k
Total hosts	N $(= r \times n)$	$k^3/4$
Total switches	$2r + k$	$5k^2/4$
Strict-sense nonblocking	$m \geq 2n - 1$	—
Rearrangeably nonblocking	$m \geq n$	—
Hosts per edge switch	—	$k/2$

2.1 Clos — Illustrative Example ($N = 12,\ n = 3,\ k = 4,\ m = 3$)

Compute $r = N/n = 12/3 = 4$.

	Stage 1 (input)	Stage 2 (middle)	Stage 3 (output)
Number of switches	$r = 4$	$k = 4$	$r = 4$
Switch size	3×4	3×3	4×3
Labels	A_1, A_2, A_3, A_4	M_1, M_2, M_3, M_4	B_1, B_2, B_3, B_4
External ports	3 inputs each (I_1–I_{12})	—	3 outputs each (O_1–O_{12})

Wiring rule: every A_i connects to every M_j (one link each), and every M_j connects to every B_t (one link each).

Blocking check: $2n - 1 = 5 > m = 3$, so this network is **rearrangeably nonblocking** only.

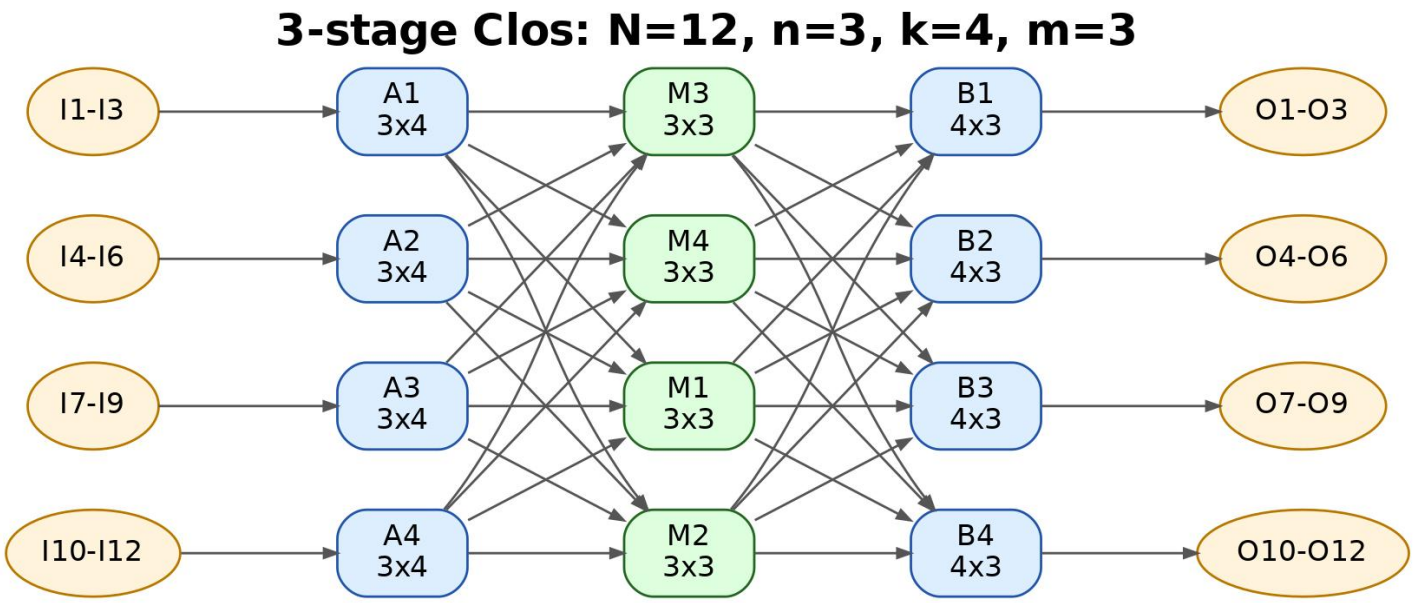


Figure 1: 3-stage Clos example: N=12, n=3, k=4, m=3

2.2 Fat-tree — Illustrative Example ($k = 4$)

Metric	Formula	Value ($k = 4$)
Pods	k	4
Core switches	$(k/2)^2$	$2^2 = 4$
Aggregation switches per pod	$k/2$	2
Edge switches per pod	$k/2$	2
Total aggregation switches	$k \times k/2$	$4 \times 2 = 8$
Total edge switches	$k \times k/2$	$4 \times 2 = 8$
Hosts per edge switch	$k/2$	2
Total hosts	$k^3/4$	$64/4 = 16$
Total switches	$5k^2/4$	$5 \times 16/4 = 20$

Wiring rule: each core switch connects to one aggregation switch per pod; each aggregation switch connects to all edge switches in its pod; each edge switch connects to $k/2$ hosts.

Exam trick: if your answer does not satisfy hosts $= k^3/4$, your wiring or switch count is wrong.

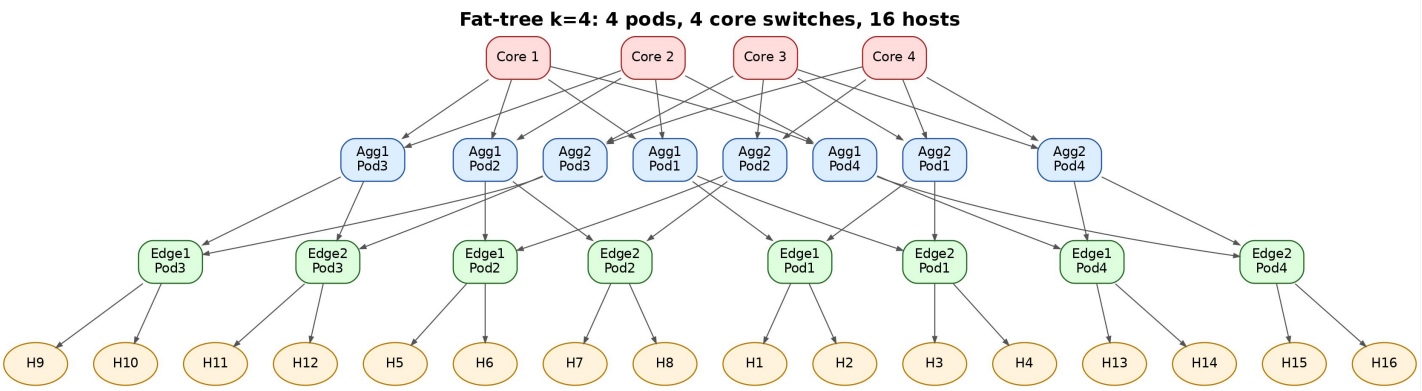


Figure 2: Fat-tree topology diagram (k=4)

3) RAID and Erasure Coding

3.1 How each RAID works

RAID	How it works	What happens on Failure	Performance Notes
RAID 0	Block-level striping across all n disks with no parity or mirroring.	Any single disk failure causes array-level data loss.	Highest read/write throughput; no redundancy overhead.
RAID 1	Mirroring: each data block is duplicated on a partner disk.	One disk in a mirror pair can fail without data loss; failure of both disks in the same pair loses data.	Fast reads (can read from either mirror), slower writes (must update both mirrors).
RAID 5	Striping with distributed single parity across disks.	Survives one disk failure; enters degraded mode and rebuilds from parity + surviving blocks.	Good read performance; write penalty from parity update (read-modify-write).
RAID 6	Striping with distributed dual parity (for example P+Q).	Survives up to two disk failures; rebuild remains possible after a second failure.	Better fault tolerance than RAID 5; higher write overhead due to dual-parity updates.

3.2 Quick formulas and tradeoffs (small p)

Assume n disks with independent disk-failure probability p, disk size S, and RAID 1 using mirrored pairs.

RAID	Usable Capacity	Storage Efficiency	Fault Tolerance	Failure Probability (small p approximation)	Write Penalty
RAID 0	nS	1	0 disk failures	$P_{loss} = 1-(1-p)^n \approx np$	1 I/O (no parity/mirror overhead)
RAID 1	$(n/2)S$	1/2	1 disk failure per mirror pair	$P_{loss} \approx (n/2)p^2$	2 I/Os (write both mirrors)
RAID 5	$(n-1)S$	$(n-1)/n$	1 disk failure	$P_{loss} \approx \binom{n}{2}p^2$	4 I/Os (read old data + read old parity + write new data + write new parity)
RAID 6	$(n-2)S$	$(n-2)/n$	2 disk failures	$P_{loss} \approx \binom{n}{3}p^3$	6 I/Os (small random write with dual parity update)

Exam takeaway: - More redundancy decreases failure probability order (from p to p^2 to p^3). - More redundancy reduces usable capacity and increases write penalty.

3.3 XOR Parity Idea and Recovery Equation

Core idea: RAID 5/6 uses XOR ($\oplus$) as the parity function because it is self-inverse — XORing a value twice cancels out, so any one missing term can be recovered from the rest.

Concept	Formula	Meaning
Parity computation	$P = D_1 \oplus D_2 \oplus \cdots \oplus D_{n-1}$	Parity block is the XOR of all data blocks in a stripe.
Recovery of failed disk D_i	$D_i = P \oplus D_1 \oplus \cdots \oplus D_{i-1} \oplus D_{i+1} \oplus \cdots \oplus D_{n-1}$	XOR all surviving blocks including P; the failed block is reconstructed.
Why XOR works	$x \oplus x = 0$ and $x \oplus 0 = x$	Double-application cancels, isolating the missing term.
RAID 6 dual parity	Two independent parity symbols P and Q	Q uses a different generator (Galois field multiplication), allowing recovery of any two simultaneous failures.

Illustrative Example — RAID 5 with 4 disks (D_1, D_2, D_3, P)

Stripe values (in bits for simplicity): $D_1 = 1100$, $D_2 = 1010$, $D_3 = 0110$.

Step 1 — Compute parity:

$$P = D_1 \oplus D_2 \oplus D_3 = 1100 \oplus 1010 \oplus 0110 = 0000$$

Disk	Value
D_1	1100
D_2	1010
D_3	0110
P	0000

Step 2 — D_2 fails. Recover using remaining disks:

$$D_2 = P \oplus D_1 \oplus D_3 = 0000 \oplus 1100 \oplus 0110 = 1010\checkmark$$

Step 3 — Verify: recovered value 1010 matches original $D_2 = 1010$.

Key point for exam: recovery always requires reading **all** surviving disks in the stripe, which causes degraded-mode read amplification.

4) CPU Scheduling (EDF / SEDF / Credit)

4.1 Mathematical Formulas / Conditions

Formula	Expression	Interpretation
CPU Utilisation	$U = \sum_i \frac{s_i}{p_i}$	Sum of each VM's time slice s_i divided by its period p_i. Represents the fraction of CPU time demanded in total.
EDF Schedulability condition	$U \leq 1$	If total utilisation does not exceed 1, all VMs can meet their deadlines. Necessary and sufficient for EDF with implicit deadlines.
Hyperperiod	$H = \text{lcm}(p_1, p_2, \ldots, p_n)$	Smallest window after which the entire schedule repeats. Used to count CPU quanta allocated to each VM in one full cycle.

4.2 SEDF tuple

(s_i, p_i, x_i): - $x_i = 0$: non-work-conserving. - $x_i = 1$: work-conserving (can consume slack).

4.3 Credit scheduler share

Formula	Expression	Interpretation
CPU share for VM i	$share_i = \frac{w_i}{\sum_j w_j}$	Fraction of CPU time VM i receives, proportional to its weight.
Credits per accounting window	$C_i = C \cdot \frac{w_i}{\sum_j w_j}$	Absolute credits allocated to VM i each replenishment window.
Fast ratio shortcut	Weights $256 : 512 : 256 \Rightarrow$ shares $1 : 2 : 1$	Simplify weight ratios before multiplying to avoid arithmetic errors.

Illustrative Example

Setup: 2 VMs, window = 150 ms, 1 credit = 10 ms, total credits $C = 15$.

VM	Weight	Credit allocation	CPU time per window
VM1	256	$15 \times \frac{256}{768} = 5$ credits	50 ms
VM2	512	$15 \times \frac{512}{768} = 10$ credits	100 ms
Total	**768**	**15 credits**	**150 ms**

Schedule trace (first 150 ms window):

Time interval	VM running	State change
0 – 50 ms	VM1	VM1 exhausts 5 credits → becomes *over*
50 – 150 ms	VM2	VM2 exhausts 10 credits → becomes *over*
150 ms	Window replenishes	Both VMs reset to *under*; cycle repeats

VM3 arrives mid-window (e.g. at t = 60 ms) with weight 256:

New total weight $= 256 + 512 + 256 = 1024$.

VM	New credit allocation	CPU time per window
VM1	$15 \times \frac{256}{1024} \approx 3.75$ credits	~37.5 ms
VM2	$15 \times \frac{512}{1024} = 7.5$ credits	75 ms
VM3	$15 \times \frac{256}{1024} \approx 3.75$ credits	~37.5 ms

VM3 starts as *under* immediately, so it can preempt any *over* VM until its own credits run out. Ratio of CPU shares remains **1 : 2 : 1**.

5) TCP AIMD Throughput

Assume sawtooth model: one packet loss per cycle, no timeout, additive increase by 1 per RTT, multiplicative decrease by half on loss.

Step	Formula	Interpretation
Window at loss	W	Peak congestion window size when a loss is detected.
Window after MD	$W/2$	Window halved immediately on loss (multiplicative decrease).
Sawtooth cycle time	$STT = \frac{W}{2} \cdot RTT$	Time for window to grow linearly from $W/2$ back to W.
Average window	$\bar{W} = \frac{W/2 + W}{2} = \frac{3W}{4}$	Mean window size over one sawtooth cycle.
Packets per cycle	$\frac{3W^2}{8}$	Total packets sent in one sawtooth cycle (area under the sawtooth).
Loss rate from W	$p = \frac{8}{3W^2}$	One loss per cycle gives this relation between p and W.
W from p	$W = \sqrt{\frac{8}{3p}}$	Rearranging the loss rate formula to express window in terms of loss probability.
Throughput	$T \approx 1.22 \cdot \frac{MTU}{RTT\sqrt{p}}$	Final result: throughput scales inversely with $\sqrt{p}$ and RTT.

6) Cloud Security

6.1 Classical Cipher Concepts

Concept	Description	Example / Notes
Caesar cipher	Shift each letter by a fixed offset k	Offset 3: A→D, B→E
Substitution cipher	Replace each symbol with another per a fixed table	Simple but vulnerable to frequency analysis
Transposition cipher	Rearrange the order of characters without changing them	Rail fence, columnar transposition
One-time pad	XOR plaintext with a random key of equal length; perfectly secure if key is never reused	$C = M \oplus K$; security breaks if K is reused
Frequency analysis	Attack that exploits letter frequency patterns in ciphertext	Breaks monoalphabetic substitution ciphers
Kerckhoffs's principle	Security must rely only on key secrecy, not algorithm secrecy	Algorithm can be public; key must stay private

6.2 Symmetric vs Asymmetric Encryption

Aspect	Symmetric Encryption	Asymmetric Encryption
Key model	Same key used for encryption and decryption	Key pair: public key encrypts, private key decrypts (or signs)
Key distribution	Secure channel needed to share the key	Public key can be shared openly; only private key is secret
Speed	Fast; suitable for bulk data encryption	Slow; used for key exchange and signatures, not bulk data
Examples	AES, DES, 3DES, ChaCha20	RSA, Diffie-Hellman, ECC
Common cloud use	Encrypting data at rest and in transit (after key exchange)	TLS handshake, digital signatures, certificate authorities
Key length for equivalent security	Shorter keys (for example AES-128 RSA-3072 in strength)	Longer keys needed for equivalent security
Confidentiality	Yes	Yes
Authentication / Non-repudiation	No (shared key means either party could forge)	Yes — only private key holder can sign
Scalability	$O(n^2)$ keys needed for n parties (each pair needs own key)	$O(n)$ key pairs — each party needs one pair

6.3 Diffie-Hellman + RSA Workflow

Step	Diffie-Hellman Key Exchange	RSA Signature
Public parameters	Prime p, generator g	Modulus $n = pq$, public exponent e
Private parameters	Alice secret a; Bob secret b	Private exponent d where $de \equiv 1 \pmod{\phi(n)}$
Key generation	—	Choose primes p, q; compute $n = pq$, $\phi(n) = (p-1)(q-1)$; pick e with $\gcd(e, \phi(n)) = 1$; compute $d = e^{-1} \bmod \phi(n)$
Public transmission	Alice sends $A = g^a \bmod p$; Bob sends $B = g^b \bmod p$	Public key (n, e) is shared openly
Shared secret / Sign	$s = B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$	Sign: $S = M^d \bmod n$
Verification	Both sides compute the same s independently	Verify: compute $M' = S^e \bmod n$; accept if $M' = M$
Security basis	Discrete logarithm hardness	Integer factorisation hardness

6.4 Diffie-Hellman — Illustrative Example

Public parameters: $p = 23$, $g = 5$.

Step	Alice ($a = 6$)	Bob ($b = 15$)
Compute public value	$A = 5^6 \bmod 23 = 15625 \bmod 23 = 8$	$B = 5^{15} \bmod 23 = 30517578125 \bmod 23 = 19$
Send to other party	Sends $A = 8$	Sends $B = 19$
Compute shared secret	$s = B^a \bmod p = 19^6 \bmod 23 = 2$	$s = A^b \bmod p = 8^{15} \bmod 23 = 2$

Both arrive at the same shared secret $s = 2$ without ever transmitting their private keys.

6.5 RSA Signature — Illustrative Example

Key generation: primes $p = 61$, $q = 53$.

Step	Computation	Value
Modulus	$n = p \times q$	3233
Euler's totient	$\phi(n) = (p-1)(q-1) = 60 \times 52$	3120
Public exponent	Pick e with $\gcd(e, 3120) = 1$	$e = 17$
Private exponent	$d = e^{-1} \bmod \phi(n)$	$d = 2753$ (since $17 \times 2753 = 46801 = 15 \times 3120 + 1$)
Sign message $M = 65$	$S = M^d \bmod n = 65^{2753} \bmod 3233$	$S = 2790$
Verify signature	$M' = S^e \bmod n = 2790^{17} \bmod 3233$	$M' = 65$ — matches original M

7) CAP Theorem

7.1 CAP Definitions

Property	Definition	What it means in practice
Consistency (C)	Every read receives the most recent write or an error	All nodes see the same data at the same time; no stale reads
Availability (A)	Every request receives a (non-error) response, though it may not reflect the latest write	System stays operational and responsive even under failure
Partition Tolerance (P)	The system continues operating despite arbitrary message loss or delay between nodes	Network splits do not bring the system down
CAP theorem	A distributed system can guarantee at most **two** of C, A, P simultaneously	In practice, P is mandatory in networks, so the real choice is **CP vs AP**

7.2 CP vs AP Trade-off

Aspect	CP System	AP System
Sacrifices	Availability (may block or return error during partition)	Consistency (may return stale data)
Behaviour during partition	Refuse to respond until consistency is restored	Respond with possibly outdated data
Real-world examples	HBase, Zookeeper, etcd, traditional RDBMS with replication	Cassandra, DynamoDB, CouchDB, DNS
Best for	Financial transactions, inventory systems, any correctness-critical workload	Search suggestions, social feeds, caching, availability-critical UX

7.3 Consistency Models

Model	Guarantee	Strength	Example
Strong Consistency	Every read sees the most recent write; all nodes agree on the same value at all times	Strongest	etcd, Zookeeper, traditional RDBMS
Weak Consistency	No guarantee that subsequent reads will see the latest write; best-effort only	Weakest	Memcached, some real-time media streams
Eventual Consistency	Given no new writes, all replicas will converge to the same value eventually	Between strong and weak	DynamoDB, Cassandra, DNS

7.4 Exam Answer Template

During partition, cannot guarantee both full Consistency and full Availability. Partition tolerance is mandatory in real distributed systems, so the design choice is CP vs AP.

1. State the partition scenario.
2. If correctness-critical — choose CP tendency (may block or error).
3. If responsiveness-critical — choose AP tendency (allow staleness).
4. Tie to real systems:
   - ReCAPTCHA-like large-scale collection: AP-leaning.
   - Instant search suggestion: strongly AP-leaning.
   - Bank transfers or booking systems: CP-leaning.

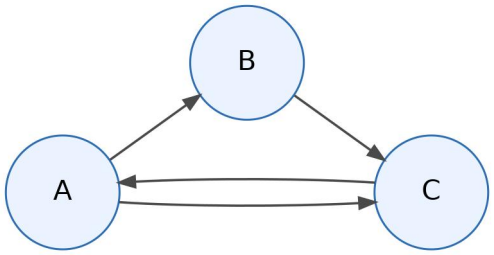
8) PageRank

8.1 Mathematical Formulas

Concept	Formula	Interpretation
Core PageRank (no teleport)	$r_i = \sum_{j \to i} \frac{r_j}{d_j}, \quad \sum_i r_i = 1$	Rank of node i is the sum of rank contributions arriving from inbound neighbors; each neighbor spreads its rank equally across its outgoing links.
Damped PageRank	$\mathbf{r} = dM\mathbf{r} + (1-d)\frac{1}{n}\mathbf{1}$	With probability d, follow a link; with probability $(1-d)$, jump uniformly to any node. Prevents sinks and disconnected components from trapping all rank.
Fast approximation from old rank	$\mathbf{r}' \approx d\mathbf{r} + (1-d)\frac{1}{n}\mathbf{1}$	Quick one-step estimate when damping changes only slightly and a full recomputation is unnecessary.
Common damping value	$d \approx 0.85$ to 0.9	Standard practical range used in many PageRank-style systems.

8.2 Illustrative Example

Consider 3 pages with links: $A \to B$, $A \to C$, $B \to C$, $C \to A$.



Out-degrees: - $d_A = 2$ - $d_B = 1$ - $d_C = 1$

Assume old rank vector is uniform:

$$\mathbf{r} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

Using one-step PageRank without teleport:

Node	Incoming contribution	New rank
A	from C: $1/3$	$r_A = 1/3$
B	from A: $(1/3)/2$	$r_B = 1/6$
C	from A: $(1/3)/2$ and from B: $1/3$	$r_C = 1/2$

So the updated rank vector is:

$$\mathbf{r}' = \begin{bmatrix} 1/3 \\ 1/6 \\ 1/2 \end{bmatrix}$$

Interpretation: page C gets the highest rank because it receives votes from both A and B, while B receives only half of A's vote.

9) Consistent Hashing (Event Trace Method)

9.1 Core rule

Place server tokens and data hashes on ring $[0, M-1]$. Each key goes to nearest clockwise server token.

9.2 Exam workflow

1. Compute all server token positions (including replicas/vnodes).
2. Sort ring clockwise.
3. For each data item, compute all replica hash positions.
4. Map each position clockwise to server.
5. Apply events in order (add server, fail server, add key).
6. Recompute only affected intervals.

9.3 Rebalance intuition

- Add server: mainly keys in predecessor arc move to new server.
- Remove server: only keys mapped to removed server move clockwise.
- Good property: localized remapping, not global remapping.

9.4 Load-balance risk formula

Probability n random points lie in some semicircle:

$$P = \frac{n}{2^{n-1}}$$

Meaning: uneven token clustering can create load skew; using many vnodes reduces this risk.

9.5 Directory-Based vs Decentralized Lookup

Model	Core idea	Advantages	Disadvantages	Typical use
Directory-based lookup	A central directory stores the mapping from keys to storage nodes; clients ask the directory first, then contact the target node	Simple design; easy to reason about; usually fast lookup path; easy to enforce global policies	Directory can become bottleneck; possible single point of failure unless replicated; metadata must be updated centrally	Small clusters, metadata-server architectures, master-based systems
Decentralized lookup	Each node computes or routes lookup using hash ring / DHT logic without a central directory	No central bottleneck; better scalability; fits consistent hashing naturally; more fault tolerant	More routing complexity; lookups may take multiple hops; harder to debug and coordinate membership changes	Peer-to-peer overlays, Dynamo-style systems, distributed hash tables

Exam point: directory-based lookup is simpler to reason about, while decentralized lookup scales better and avoids a central bottleneck.

9.6 Illustrative Example

Assume hash ring size $M = 100$ with server tokens:

- $S_1 = 5$
- $S_2 = 20$
- $S_3 = 35$
- $S_4 = 60$
- $S_5 = 85$

Data items hash to:

- $K_1 = 12$
- $K_2 = 42$
- $K_3 = 72$

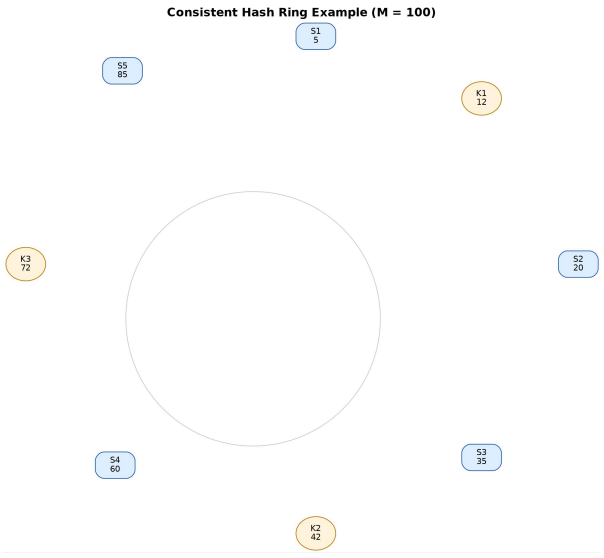


Figure 3: Consistent hashing ring example

Clockwise assignment rule: each key is placed on the first server token encountered when moving clockwise.

Key	Hash position	Clockwise successor	Assigned server
K_1	12	20	S_2
K_2	42	60	S_4
K_3	72	85	S_5

If a new server $S_6 = 50$ is added, only keys in the interval $(35, 50]$ remap to S_6.

Key	Old server	New server after adding $S_6 = 50$
$K_1 = 12$	S_2	S_2 (unchanged)
$K_2 = 42$	S_4	S_6
$K_3 = 72$	S_5	S_5 (unchanged)

Key exam point: consistent hashing causes **localized remapping**. Adding one server does not reshuffle all keys; only keys in one predecessor arc move.

10) Time Management for 2-Hour Paper

1. First 5 min: mark all calculation-heavy parts.
2. Do formula questions first (AIMD, EDF, RAID, DH/RSA, hash ring).
3. For theory questions, use compare tables and tradeoff language.
4. If stuck in long trace question, show method table first (often earns method marks).
5. Final 10 min: check arithmetic and ranking/order statements.